Linux Networking

Think of the progression like this:

Fundamentals → Linux networking → TCP/IP → Troubleshooting → Security → Cloud → Containers → Kubernetes → Advanced Platform Engineering

---

1. What Networking Actually Means

At the simplest level:

Networking is the process of allowing two or more systems to communicate with each other.

For example:

```
Your Laptop
     |
     | HTTP
     |
     v
Web Server
     |
     | TCP
     |
     v
Database Server
```

As a DevOps/Platform Engineer, you constantly deal with:

- Servers communicating with servers
- Applications communicating with databases
- Containers communicating with containers
- Kubernetes Pods communicating with Services
- Applications communicating with AWS services
- Users communicating with applications
- Load balancers forwarding traffic
- DNS resolving names to IP addresses
- Firewalls allowing/denying traffic
- TLS encrypting communication

So networking is **one of the foundations of DevOps**.

---

2. Networking Mental Model

Before learning commands, understand these concepts:

```
Application
    ↓
HTTP / HTTPS / SSH / DNS
    ↓
TCP / UDP
    ↓
```

```
IP
    ↓
Ethernet
    ↓
Network Interface
    ↓
Physical / Virtual Network
```

For example, when you execute:

```
curl https://google.com
```

many things happen.

Conceptually:

```
curl
 ↓
HTTPS
 ↓
TCP
 ↓
IP
 ↓
Network Interface
 ↓
Internet
 ↓
Google
```

You need to understand every layer.

---

3. OSI Model

You should know the OSI model because it gives you a framework for troubleshooting.

Layer	Name	Examples
7	Application	HTTP, DNS, SSH
6	Presentation	TLS, encryption
5	Session	Sessions
4	Transport	TCP, UDP
3	Network	IP, routing
2	Data Link	Ethernet, MAC, ARP
1	Physical	Cable, fiber, radio

For DevOps, the most important layers are:

```
L7 → HTTP/DNS/SSH
L4 → TCP/UDP/Ports
L3 → IP/Routing
L2 → Ethernet/MAC/ARP
```

Example incident:

Application cannot connect to database.

Don't immediately assume the application is broken.

Troubleshoot layer by layer:

```
DNS?
 ↓
IP?
 ↓
Routing?
 ↓
TCP?
 ↓
Port?
 ↓
Firewall?
 ↓
TLS?
 ↓
Application?
```

That mindset is extremely important.

---

4. IP Address

An IP address identifies a network interface on an IP network.

Example:

```
192.168.1.10
```

IPv4 contains four octets:

```
192 . 168 . 1 . 10
```

Each ranges from:

```
0 - 255
```

---

5. Private vs Public IP

Private IPv4 ranges include:

`10.0.0.0/8`

`172.16.0.0/12`

`192.168.0.0/16`

Example:

```
10.0.1.25
192.168.1.50
172.16.10.20
```

These are commonly used inside:

- Corporate networks
- AWS VPCs
- Docker networks
- Kubernetes networks
- Home networks

A public IP is reachable through the public Internet subject to routing and security controls.

---

6. Linux Network Interface

A Linux server can have interfaces such as:

```
eth0
ens5
ens33
lo
```

Check them:

```
ip link
```

or:

```
ip addr
```

Example:

```
eth0
    inet 10.0.1.20/24
```

This tells you that the interface has:

```
IP      = 10.0.1.20
Prefix  = /24
```

---

7. The `ip` Command

Modern Linux networking relies heavily on the `ip` command.

You should become very comfortable with:

```
ip addr
ip link
ip route
ip neigh
```

Show IP addresses

```
ip addr
```

Short form:

```
ip a
```

Show interfaces

```
ip link
```

Show routing table

```
ip route
```

Show ARP/neighbour information

```
ip neigh
```

These four commands alone are extremely important for Linux troubleshooting.

---

8. Loopback Interface

You will commonly see:

```
lo
```

with:

```
127.0.0.1
```

This is the loopback interface.

Example:

```
ping 127.0.0.1
```

or:

```
ping localhost
```

Traffic stays within the machine.

This is extremely important when troubleshooting applications.

For example:

```
Application listening on 127.0.0.1:8080
```

means the application is only accessible locally.

It may NOT be accessible from another server.

Compare:

```
127.0.0.1:8080
```

with:

`0.0.0.0:8080`

The second generally means the application is listening on all IPv4 interfaces.

---

9. Ports

An IP identifies the machine/interface.

A port identifies the service/process endpoint.

Think:

`IP + Port`

Example:

`10.0.1.20:8080`

Common ports:

Port	Protocol	Typical use
22	TCP	SSH
53	TCP/UDP	DNS
80	TCP	HTTP
443	TCP	HTTPS
3306	TCP	MySQL
5432	TCP	PostgreSQL
6379	TCP	Redis
8080	TCP	Applications
9090	TCP	Prometheus
3000	TCP	Grafana

---

10. Listening Ports

One of your most important troubleshooting questions is:

Is the application actually listening?

Use:

```
ss -lnt
```

More useful:

```
ss -lntp
```

Meaning:

```
-l = listening
-n = don't resolve names
-t = TCP
-p = process
```

For UDP:

```
ss -lnup
```

Everything:

```
ss -lntup
```

Example:

```
LISTEN 0 128 0.0.0.0:8080
```

This tells you something is listening on port 8080.

---

11. TCP vs UDP

This is fundamental.

TCP

TCP provides:

- Connection-oriented communication
- Reliability
- Ordering
- Retransmission
- Flow control

Used by:

```
HTTP
HTTPS
SSH
MySQL
PostgreSQL
```

UDP

UDP is connectionless and lightweight.

Used commonly by:

```
DNS
DHCP
streaming
certain real-time applications
```

A simplified comparison:

```
TCP
Client ——— connection ——— Server
       reliable

UDP
Client ——— packet ————— Server
       lightweight
```

---

12. TCP Three-Way Handshake

You absolutely need to understand this.

When a TCP connection is established:

```
Client                    Server

  SYN ------------------->

      <---------------- SYN-ACK

  ACK ------------------->
```

Then the connection is established.

This becomes extremely useful when diagnosing:

```
Connection refused
Connection timeout
Connection reset
```

These are **not the same problem**.

---

13. Connection Refused vs Timeout

This is a critical DevOps troubleshooting distinction.

Connection refused

Example:

```
curl 10.0.1.20:8080
```

returns:

```
Connection refused
```

Often means:

```
Host reachable
        ↓
Port reached
        ↓
Nothing listening
```

or a firewall is actively rejecting the connection.

Connection timeout

Usually indicates something like:

```
Routing problem
Firewall/security group
Network ACL
Network path
Dropped packets
```

Conceptually:

```
Connection refused

Client → Server
          X
        REJECT
```

versus:

```
Timeout

Client → → → → Server
        packets dropped
        ...............
```

This distinction saves enormous troubleshooting time.

---

14. DNS

DNS converts names into IP addresses.

For example:

```
google.com
      ↓
IP address
```

Linux DNS troubleshooting commands:

```
dig google.com
nslookup google.com
getent hosts google.com
```

You should understand:

```
A
AAAA
CNAME
MX
TXT
NS
PTR
```

Eventually you should understand:

```
DNS client
    ↓
Resolver
    ↓
Root
    ↓
TLD
    ↓
Authoritative DNS
```

---

15. /etc/resolv.conf

Linux DNS configuration commonly involves:

```
cat /etc/resolv.conf
```

You may see:

```
nameserver 10.0.0.2
```

This tells the system where to send DNS queries.

But modern Linux systems may use:

```
systemd-resolved
NetworkManager
```

so you should understand the actual resolver architecture rather than memorizing one file.

---

16. Routing

Routing answers:

Where should this packet go?

Check:

```
ip route
```

Example:

```
default via 10.0.1.1 dev eth0
10.0.1.0/24 dev eth0
```

Meaning:

```
10.0.1.0/24
     ↓
eth0
```

And:

```
default
   ↓
```

```
10.0.1.1
```

Anything without a more specific route goes through the default gateway.

---

17. Default Gateway

Suppose:

```
Server:
10.0.1.20

Gateway:
10.0.1.1
```

The server can communicate directly with hosts on its local subnet.

For another network:

```
10.0.2.20
```

it sends traffic toward:

```
10.0.1.1
```

The gateway then forwards it.

---

18. CIDR and Subnets

You must become very comfortable with:

```
/8
/16
/24
/25
/26
/27
/28
/29
/30
```

Example:

```
192.168.1.0/24
```

Typically provides 256 IPv4 addresses:

```
192.168.1.0
...
192.168.1.255
```

Traditional IPv4 host allocation gives:

```
Network address:
192.168.1.0
```

```
Usable range:
192.168.1.1 - 192.168.1.254

Broadcast:
192.168.1.255
```

You need subnetting because it appears everywhere:

```
AWS VPC
Kubernetes
Docker
VPN
datacenters
routing
firewalls
```

---

19. ARP

ARP maps:

```
IP → MAC address
```

For IPv4 local-network communication.

Check neighbours:

```
ip neigh
```

Example:

```
10.0.1.1 dev eth0 lladdr aa:bb:cc:dd:ee:ff REACHABLE
```

This tells Linux:

```
10.0.1.1
   ↓
MAC address
```

---

20. Network Diagnostics

Your core toolkit should include:

```
ping
curl
wget
ss
ip
dig
nslookup
traceroute
tracepath
nc
tcpdump
```

For example:

Test reachability

```
ping 10.0.1.20
```

Test HTTP

```
curl -v http://10.0.1.20:8080
```

Test TCP port

```
nc -vz 10.0.1.20 8080
```

Check DNS

```
dig example.com
```

Check route

```
ip route
```

Inspect packets

```
sudo tcpdump -i eth0
```

---

21. `curl` — Extremely Important

For DevOps, `curl` is one of the most useful networking tools.

Start with:

```
curl http://example.com
```

Headers:

```
curl -I http://example.com
```

Verbose:

```
curl -v http://example.com
```

Specific port:

```
curl http://10.0.1.20:8080
```

HTTPS:

```
curl -v https://example.com
```

You should eventually be able to understand the output:

```
DNS
 ↓
TCP
 ↓
TLS
 ↓
```

`HTTP`

---

22. `nc` — Netcat

Netcat is excellent for testing connectivity.

Example:

```
nc -vz 10.0.1.20 8080
```

If successful:

```
Connection succeeded
```

This answers:

Can I establish a TCP connection to this port?

That's different from:

Does the application return a valid HTTP response?

Therefore:

`nc`

tests connectivity.

`curl`

can test application-layer HTTP.

---

23. `tcpdump`

This is where you start moving toward advanced troubleshooting.

Example:

```
sudo tcpdump -i eth0
```

Capture TCP port 8080:

```
sudo tcpdump -i eth0 tcp port 8080
```

Capture traffic from a host:

```
sudo tcpdump -i eth0 host 10.0.1.20
```

Capture DNS:

```
sudo tcpdump -i eth0 port 53
```

You should eventually learn to identify:

```
SYN
SYN-ACK
ACK
FIN
RST
retransmissions
```

from packet captures.

---

24. Firewalls

Linux systems may use:

```
nftables
iptables
firewalld
ufw
```

On RHEL-based systems, you should especially understand:

```
firewall-cmd
```

and the underlying:

```
nftables
```

Check firewall state:

```
sudo firewall-cmd --state
```

List configuration:

```
sudo firewall-cmd --list-all
```

For modern Linux networking, learn **nftables concepts**, even if you encounter iptables-compatible commands in older environments.

---

25. NAT

NAT = Network Address Translation.

A common example:

```
Private Network

10.0.1.20
    |
    | NAT
    v
Public IP
    |
 Internet
```

This is fundamental to:

```
AWS
Docker
Kubernetes
home networks
VPNs
```

You should understand:

```
SNAT
DNAT
MASQUERADE
```

---

26. Linux Networking and Containers

Now networking becomes directly relevant to DevOps.

Docker creates virtual networking.

Typical architecture:

```
Container
    |
    v
Virtual Ethernet
    |
    v
Docker bridge
    |
    v
Host
    |
    v
Network
```

Useful commands:

```
docker network ls
docker network inspect bridge
```

You should understand:

```
bridge
host
none
custom bridge
port mapping
NAT
container DNS
```

For example:

```
docker run -p 8080:80 nginx
```

means:

```
Host :8080
     ↓
Container :80
```

27. Kubernetes Networking

Once Linux networking is strong, Kubernetes becomes much easier.

You need to understand:

```
Pod IP
Service IP
ClusterIP
Node IP
NodePort
LoadBalancer
Ingress
CoreDNS
CNI
kube-proxy
NetworkPolicy
```

The conceptual flow:

```
User
 ↓
Load Balancer
 ↓
Ingress
 ↓
Service
 ↓
Pod
```

And inside the cluster:

```
Pod
 ↓
CNI
 ↓
Node networking
 ↓
Cluster network
```

This is why I would **not** recommend jumping directly into Kubernetes networking before understanding Linux networking.

---

28. Cloud Networking

For AWS Platform Engineering, map Linux networking concepts to AWS:

Linux concept	AWS equivalent/concept
Interface	ENI
IP address	Private/Public IP
Routing	Route Table
Gateway	Internet/NAT Gateway
Firewall	Security Group/NACL

Linux concept	AWS equivalent/concept
Network	VPC
Subnet	VPC Subnet
DNS	Route 53
Load balancing	ALB/NLB
VPN	Site-to-Site VPN
NAT	NAT Gateway

Eventually you should be able to troubleshoot:

```
EC2
 ↓
Security Group
 ↓
Route Table
 ↓
NACL
 ↓
Subnet
 ↓
NAT/IGW
 ↓
Internet
```

---

29. Advanced Networking Topics

After the fundamentals, move into:

TCP internals

```
Window size
Congestion control
Retransmission
TIME_WAIT
CLOSE_WAIT
SYN backlog
Ephemeral ports
MTU
MSS
```

Linux internals

```
Network namespaces
veth pairs
Linux bridge
routing tables
iptables/nftables
conntrack
```

Performance

```
latency
throughput
packet loss
jitter
RTT
bandwidth
```

```
TCP retransmissions
socket exhaustion
```

Advanced tools

```
ss
tcpdump
ip
ethtool
nstat
sar
iperf3
dig
tracepath
mtr
```

---

30. Network Namespaces

This is especially important for:

```
Docker
Kubernetes
CNI
containers
```

A network namespace provides an isolated networking environment.

Conceptually:

```
Host Network Namespace
          |
          V
          eth
          |
          v
Container Network Namespace
```

You can explore this directly with:

```
ip netns
```

and:

```
ip netns exec <namespace> ip addr
```

Understanding network namespaces will make Kubernetes networking much easier.

---

31. The Professional Troubleshooting Framework

This is what I want you to eventually internalize.

Suppose:

```
Application A
    |
    X
    |
Database B
```

Don't randomly execute commands.

Follow this sequence.

Step 1 — DNS

```
dig database.example.com
```

Step 2 — IP

```
getent hosts database.example.com
```

Step 3 — Routing

```
ip route
```

Step 4 — Reachability

```
ping <IP>
```

Step 5 — Port

```
nc -vz <IP> 5432
```

Step 6 — Socket state

```
ss -ant
```

Step 7 — Application protocol

```
curl -v ...
```

Step 8 — Firewall

Check:

```
firewalld
nftables
Security Group
NACL
```

Step 9 — Packet capture

```
tcpdump
```

Step 10 — Application

Only after the network path is proven should you investigate the application itself.

---

32. Your Linux Networking Learning Progression

I recommend this order:

```
LEVEL 1 — Fundamentals
│
├── What is networking?
├── OSI model
├── TCP/IP model
├── IPv4
├── IPv6
├── MAC
├── Ports
└── Protocols
        ↓
LEVEL 2 — Linux Networking
│
├── ip
├── ss
├── ping
├── curl
├── nc
├── dig
├── traceroute
└── tcpdump
        ↓
LEVEL 3 — Network Architecture
│
├── Subnets
├── CIDR
├── Routing
├── Gateway
├── ARP
├── DNS
├── DHCP
└── NAT
        ↓
LEVEL 4 — Security
│
├── firewalld
├── nftables
├── iptables
├── ports
├── ACLs
└── Network security
        ↓
LEVEL 5 — Troubleshooting
│
├── DNS failures
├── timeout
├── refused
├── reset
├── packet loss
├── latency
└── tcpdump
        ↓
LEVEL 6 — Containers
│
├── namespaces
├── veth
├── bridge
├── NAT
└── Docker networking
        ↓
```

```
LEVEL 7 — Kubernetes
│
├── Pod networking
├── Services
├── kube-proxy
├── CoreDNS
├── CNI
└── NetworkPolicy
        ↓
LEVEL 8 — Cloud
│
├── VPC
├── Subnets
├── Route tables
├── SG
├── NACL
├── NAT Gateway
├── ALB/NLB
└── Private connectivity
        ↓
LEVEL 9 — Advanced
│
├── TCP internals
├── MTU/MSS
├── conntrack
├── performance
├── packet analysis
└── network architecture
```

33. The Most Important Practical Labs

Don't learn this only theoretically.

You should build labs such as:

Lab 1 — Two Linux machines

```
Linux A ⟷ Linux B
```

Practice:

```
ping
ssh
ip
ss
nc
tcpdump
```

Lab 2 — HTTP server

Run nginx:

```
sudo systemctl start nginx
```

Find its port:

```
ss -lntp
```

Test:

```
curl localhost
```

Then access it from another machine.

Lab 3 — Break the network

Intentionally:

```
stop service
block port
remove route
break DNS
change listening address
```

Then troubleshoot it.

This is **far more valuable** than simply memorizing commands.

Lab 4 — Docker networking

Create:

```
Container A
Container B
```

Make them communicate.

Then investigate:

```
docker network inspect
```

Lab 5 — Kubernetes networking

Build:

```
Pod → Service → Pod
```

Then investigate:

```
kubectl get pods -o wide
kubectl get svc
kubectl get endpoints
```

and correlate Kubernetes networking with Linux networking.

---

34. What "Advanced" Means for a DevOps Engineer

I would consider you strong in Linux networking when you can receive an incident like:

"The application is running, but users are getting intermittent connection timeouts."

and systematically determine whether the issue is:

```
DNS
 ↓
Application
 ↓
Socket
 ↓
TCP
 ↓
Routing
 ↓
Firewall
 ↓
NAT
 ↓
Load Balancer
 ↓
Kubernetes
 ↓
Cloud networking
```

rather than trying random commands.

That's the level we're aiming for.

Your First Lesson

I recommend that we **do not cover everything at once**.

We'll learn it like an actual professional training program.

Linux Networking — Module 1

Start with:

1. What is a network?
2. Client vs server
3. IP address
4. IPv4
5. IPv6
6. Private vs public IP
7. MAC address
8. Network interface
9. Ports
10. Protocols
11. TCP vs UDP
12. OSI model
13. TCP/IP model
14. Hands-on Linux commands
15. Troubleshooting exercises